Practical 4

Aim: Programs on lambdas and events

4.1.1 Write a program to add two numbers using a lambda expression.

```
using System;
namespace LambdaDemo
{
   class Program
   {
      static void Main(string[] args)
      {
         Func<int, int, int> add = (a, b) => a + b;
         Console.WriteLine("Enter First Number:");
         int x = Convert.ToInt32(Console.ReadLine());
         Console.WriteLine("Enter Second Number:");
         int y = Convert.ToInt32(Console.ReadLine());
         Console.WriteLine("Sum = " + add(x, y));
         Console.ReadKey();
      }
   }
}
```

4.1.2 Write a program to calculate the square of a number using a lambda expression.

```
using System;
namespace LambdaDemo
{
   class Program
   {
      static void Main(string[] args)
      {
         Func<int, int> square = n => n * n;

         Console.Write("Enter Number: ");
         int num = Convert.ToInt32(Console.ReadLine());

         Console.WriteLine("Square = " + square(num));
         Console.ReadKey();

      }
   }
}
```

4.1.3 Check whether a number is even or odd using a lambda expression.

```
using System;
class Program
{
   static void Main()
   {
      Predicate<int> isEven = n => n % 2 == 0;

      Console.Write("Enter Number: ");
      int num = Convert.ToInt32(Console.ReadLine());

      if (isEven(num))
         Console.WriteLine("Even Number");
      else
         Console.WriteLine("Odd Number");
         Console.ReadKey();
   }
}
```

4.1.4 Sort a list using lambda expressions.

```
using System;
using System.Collections.Generic;
class Program
{
   static void Main()
   {
      List<string> students = new List<string>()
      {
         "Riya",
         "Ankit",
         "Vikas",
         "Priya"
      };

      students.Sort((x, y) => x.CompareTo(y));
      Console.WriteLine("Sorted Names:");
      foreach (string s in students)
      {
         Console.WriteLine(s);
      }
Console.ReadKey();
   }
}
```

4.2.1 Create and raise an event in C#.

```
using System;
class Alarm
{
```

```
    public event Action Ring;

    public void Start()
    {
        Console.WriteLine("Alarm Started...");

        Ring?.Invoke();
    }
}

class Program
{
    static void Main()
    {
        Alarm alarm = new Alarm();

        alarm.Ring += () =>
        {
            Console.WriteLine("Wake Up! Alarm is Ringing.");
        };

        alarm.Start();
        Console.ReadKey();
    }
}
```

4.2.2: Implement an event using the EventHandler delegate.

```
using System;
class Student
{
    public event EventHandler ResultDeclared;

    public void DeclareResult()
    {
        Console.WriteLine("Result Declared");

        ResultDeclared?.Invoke(this, EventArgs.Empty);
    }
}

class Program
{
    static void Main()
    {
        Student s = new Student();
        s.ResultDeclared += ShowResult;
        s.DeclareResult();
    }
```

```
    static void ShowResult(object sender, EventArgs e)
    {
        Console.WriteLine("Congratulations! Check Your Result.");
            Console.readline();
    }
}
```

4.2.3 Handle an event using a lambda expression.

```
using System;
class Notification
{
    public event Action MessageSent;

    public void Send()
    {
        Console.WriteLine("Sending Message...");

        MessageSent?.Invoke();
    }
}

class Program
{
    static void Main()
    {
        Notification notification = new Notification();
        notification.MessageSent += () =>
        {
            Console.WriteLine("Message Sent Successfully.");
        };

        notification.Send();
            Console.ReadKey();
    }
}
```